CLEAN CODE

HERAUSFORDERUNGEN IN DER SOFTWAREENTWICKLUNG

- 📚 Komplexität von Software
- ⏳ Wartbarkeit und Erweiterbarkeit
- 💰 Technical Debt und steigende Kosten
- 🐞 Unklare oder widersprüchliche Anforderungen
- 🧹 Nachträgliches Bereinigen und Aufräumen von Quellcode (Refactoring)

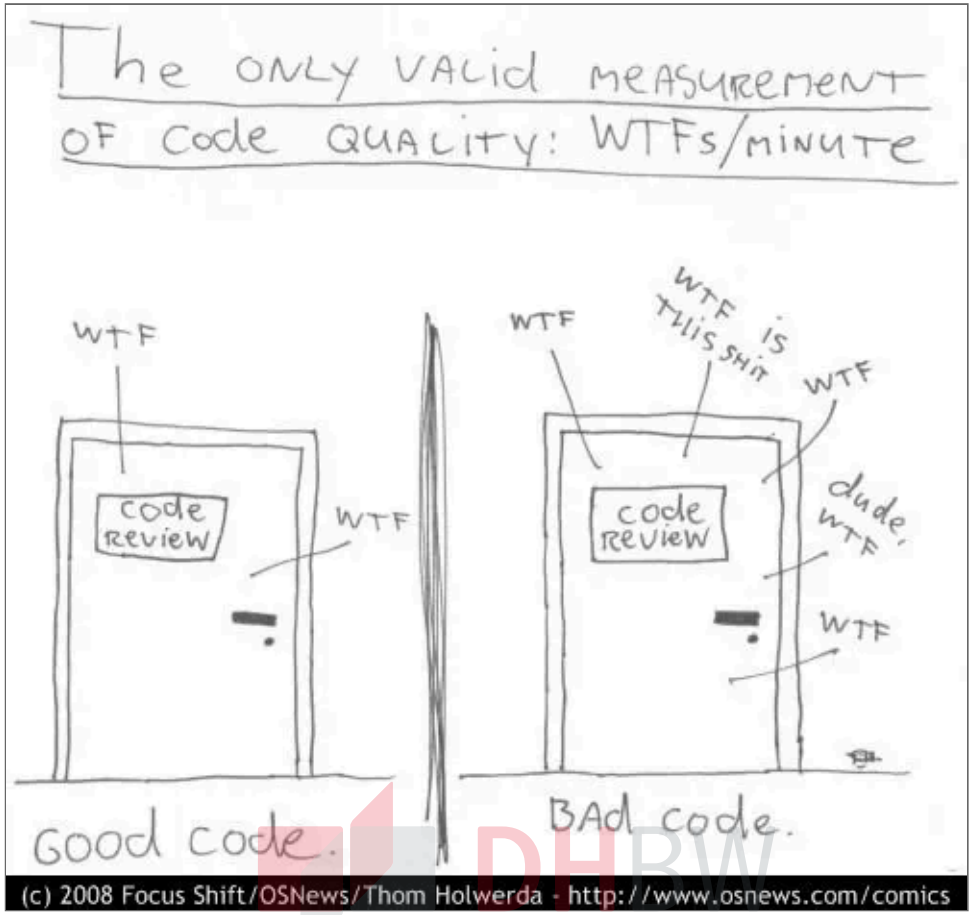
SOFTWARE QUALITÄT

Software Qualität umfasst verschiedene Aspekte, die sicherstellen, dass eine Softwarelösung den Anforderungen entspricht und zuverlässig funktioniert. Dazu gehören:

Qualitätsmerkmal	Beschreibung
✅ **Funktionalität**	Erfüllt die Software die Anforderungen?
⚙️ **Zuverlässigkeit**	Wie oft treten Fehler auf?
🏃 **Performance**	Wie schnell reagiert die Software?
🔒 **Sicherheit**	Wie gut ist die Software gegen Angriffe geschützt?
🧹 **Wartbarkeit**	Wie einfach ist es, den Code zu verstehen, anzupassen und zu erweitern?
🔄 **Testbarkeit**	Wie leicht lassen sich Tests schreiben und ausführen?

👉 **ISO 25010**: Ein internationaler Standard, der Qualitätsmerkmale für Software definiert.

WAS IST CLEAN CODE?



WAS IST CLEAN CODE?

🎯 *"Clean code does one thing well."*

- Bjarne Stoustrup, inventor of C++

📖 *"Clean code is simple and direct. Clean code reads like well-written prose."*

- Grady Booch, author of Object Oriented Analysis

🤝 *"Clean code can be read, and enhanced by a developer other than its original author."*

- Dave Thomas, godfather of the Eclipse strategy

🧹 *"Clean code always looks like it was written by someone who cares."*

- Michael Feathers, author of Working with Legacy Code

🤥 *"Code never lies, comments sometimes do"*

- *Ron Jeffries, author of Extreme Programming books*

Quelle: Clean Code by Robert C. Martin

WIE VERBRINGEN ENTWICKLER IHRE ZEIT?

R. Minelli, A. Mocci, and M. Lanza, “I know what you did last summer - an investigation of how developers spend their time,” in *23rd ICPC*, 2015.

X. Xia, L. Bao, D. Lo, Z. Xing, A. E. Hassan, and S. Li, “Measuring program comprehension: A large-scale field study with professionals,” *IEEE Transactions on Software Engineering*, vol. 44, no. 10, pp. 951-976, 2017.

👉 Entwickler verbringen einen Großteil ihrer Zeit (bis zu 60%) mit dem Lesen und Verstehen von Code.

READABILITY

Lesbarkeit ist ein entscheidendes Merkmal von Clean Code. Gut lesbarer Code erleichtert es Entwicklern, den Code zu verstehen, zu warten und zu erweitern. Wichtige Aspekte der Lesbarkeit sind:

- 📖 Klar und verständlich
- 🧩 Gut strukturiert und organisiert
- 🔤 Aussagekräftige Namen für Variablen, Funktionen und Klassen
- 📏 Konsistente Formatierung und Einrückung
- 💬 Angemessene Kommentare zur Erklärung von komplexem Code

READABILITY - NAMING

Was macht der folgende Code?

```
1  public double discP(double amt, int cStat, int y) {
2    double p = 0.0;
3
4    if (cStat == 1) {
5      p = amt * 0.05;
6    } else if (cStat == 2 && y > 2) {
7      p = amt * CUtils.specD;
8    } else if (cStat == 3) {
9      double specDisPr = amt * 0.15;
10     if (amt > 1000) specDisPr += 50;
11     p = specDisPr;
12   }
13   return p;
14 }
```

READABILITY - NAMING

Was macht der folgende Code?

```
1    public double calculateDiscountPrice(double purchaseAmount, int
   customerStatus, int yearsAsCustomer) {
2    double price = 0.0;
3
4    if (customerStatus == 1) {
5      price = purchaseAmount * 0.05;
6    } else if (customerStatus == 2 && yearsAsCustomer > 2) {
7      price = purchaseAmount * Customer.SILVER_CUSTOMER_DISCOUNT;
8    } else if (customerStatus == 3) {
9      double goldCustomerDiscount = purchaseAmount * 0.15;
10     if (purchaseAmount > 1000) goldCustomerDiscount += 50;
11     price = goldCustomerDiscount;
12   }
13
14   return price;
15 }
```

READABILITY - NAMING CONVENTIONS

- 🔤 Verwende aussagekräftige und beschreibende Namen
- 🌍 Verwende englische Namen für bessere internationale Zusammenarbeit
- 📐 Gib Einheiten in Variablennamen an (z.B. `delayInSeconds`, `weightInKg`, `priceInEuros`)
  Oder noch besser: Verwende Typen, die Einheiten repräsentieren (z.B. `Duration`, `Weight`, `Money`)
- 🏃 Verwende Verben für Funktionen/Methoden (z.B. `calculateTotal`, `validateInput`)
- 📦 Verwende Nomen für Klassen/Objekte (z.B. `Customer`, `OrderProcessor`)
- 🎯 Ein Wort pro Konzept (z.B. `fetch` ODER `retrieve` ODER get - nicht alle drei)
- 📏 Keine Angst vor langen Namen! Moderne IDEs nutzen code completion
- 🧩 Vermeide Abkürzungen, es sei denn, sie sind allgemein bekannt
- 🐫 Verwende konsistente Namenskonventionen (z.B. camelCase, PascalCase)
- 🚫 Vermeide Noise Words (z.B. `Data`, `Utils`, `Info`, `Manager`, `Helper`)
- 🗣️ Namen sollten aussprechbar sein (z.B. `OcmTx` ist schlecht)
- 🔍 Namen sollten suchbar sein (z.B. `i` oder `7` sind schlechte Namen)

NAMING CONVENTIONS - ÜBUNG

Gibt es Verbesserungsbedarf bei den folgenden Namen?

Code	💡
`int d;`	❌
`int dayOfTheMonth;`	✅
`class PerformAgeCheck {...}`	❌
`public void awaitConfirmation(int maxWaitTime) {...}`	❌
`public void validateInput(String input) {...}`	✅
`class TextActionsHelper {...}`	❌
`List<User> users;`	✅

READABILITY - FORMATTING CONVENTIONS

```
1  boolean isAllowedToDrive(Person person) {
2    if (person.getAge() >= 18)
3    {
4      if (person.hasValidLicense()) {
5        if(person.isSober()) {
6          Logger.log("The Person is sober has a valid drivers license and is
   therefore by all means allowed to drive.");
7          return true;
8        } else {
9          Logger.log("Person is not sober.");
10         return false;
11       }
12     } else {
13       Logger.log("Person does not have a license.");
14       return false;
15     }
16   }
17   else
18   {
19     Logger.log("Person is underaged."); return false;
20   }
```

```
21 }
```

READABILITY - FORMATTING CONVENTIONS

```java
boolean isAllowedToDrive(Person person) {
  if (person.getAge() < 18) {
    Logger.log("Person is underaged.");
    return false;
  }
  if (!person.hasValidLicense()) {
    Logger.log("Person does not have a valid license.");
    return  false;
  }
  if (!person.isSober()) {
    Logger.log("Person is not sober.");
    return false;
  }
  return true;
}
```

READABILITY - FORMATTING CONVENTIONS

- Konsistente Einrückung und Formatierung
- Eine Anweisung pro Zeile für bessere Lesbarkeit
- Begrenze Zeilenlänge (z.B. 80-120 Zeichen)
- Vermeide zu tiefe Verschachtelung (max. 3-4 Ebenen)
- Nutze Leerzeilen zur Strukturierung von Codeblöcken
- Konsistente Klammerstil (z.B. K&R, Allman)
- Konsistente Verwendung von Leerzeichen (z.B. um Operatoren, nach Kommas)
- Verwende Code-Formatierungswerkzeuge (z.B. Prettier, Black, clang-format)
- Nutze Linting-Tools zur automatischen Überprüfung von Code-Standards (z.B. ESLint, Pylint, Checkstyle)
- Befolge etablierte Styleguides (z.B. **Google Java Style Guide**, PEP 8 für Python)

READABILITY - KOMMENTARE

```java
/**
* Returns the day of the month.
* @return the day of the month (1-31)
*/
public int getDayOfMonth() {
  return dayOfMonth; // returns the day of the month
}
```

```java
// customer must be older than 18 and either a customer for longer than 5 years
or possess a pro membership
return (customer.getYearsAsCustomer() > 5 || customer.hasProMembership())
    && customer.getAge() > 18;
```

READABILITY - KOMMENTARE

```java
public int getDayOfMonth() {
  return dayOfMonth;
}
```

```java
boolean isLongTimeCustomer = customer.getYearsAsCustomer() > 5;
boolean isAdult = customer.getAge() > 18;
boolean hasProMembership = customer.hasProMembership();
return isAdult && (isLongTimeCustomer || hasProMembership);
```

READABILITY - KOMMENTARE

- 💬 Kommentare sollten den Code erklären, nicht wiederholen!
- ⚠️ Kommentare Lügen, Code nicht
- 🔄 Halte Kommentare aktuell und konsistent mit dem Code
- 📄 Nutze Dokumentationskommentare für öffentliche APIs (z.B. Javadoc, docstrings)
- ❗ Vermeide "to-do" Kommentare - nutze stattdessen Issue-Tracker
- ✅ Kommentiere Annahmen, Einschränkungen und bekannte Probleme